

Data cleaning

Group members:

CHLOE COWARD - CMC554

JAY ZHENG - JZ2384

LILLIAN GE - LJG244

```
In [5]: # Import libraries that need, all library imports here
import pandas as pd
import numpy as np
import glob
import os
```

2. Data collection & Cleaning

Below two sub-sections describe our **data collection** and **data cleaning** process

2.1 Data Collection

Summary:

Here we bring in both data sources we'll be analyzing throughout the project. We made imported data into 2 data frames and checked the heads and tails:

- **Manual dataset (df_manual)** Chloe's self recorded resale log (2020~2025). Includes full financial info (purchase cost, platform fees, profit).
- **Depop dataset (df_depop)** Exports directly from Depop's seller dashboard. Includes platform-side info like buyer names, listing/sale dates, and bundle flags.

```
In [6]: # import Chloe's manually collected data from Oct 2020 - Oct 2025
df_manual = pd.read_csv("uncleanedData/Chloe_Manual_Sales_Data.csv")

# imported Depop released data, aggregated all files into 1 dataframe
path = "uncleanedData/DEPOP_CSV/*.csv"
filenames = glob.glob(path)

dfs = []
for filename in filenames:
    dfs.append(pd.read_csv(filename))

df_depop = pd.concat(dfs, ignore_index=True)

print("Shape of df_manual:", df_manual.shape)
print("Shape of df_depop:", df_depop.shape)
```

Shape of df_manual: (7971, 33)
Shape of df_depop: (6453, 34)

Since Depop only provides max 3 month exports at once, we loop through and merge all CSVs into one complete dataframe. This ensures both datasets cover the full time period (Q4 2020 - Q3 2025) for consistency

2.2 Cleaning

2.2.0 Aligning manual Depop data with exported data

Chloe's manually collected datasets **includes sales from other platforms** like Poshmark and Mercari but our analysis compares **only Depop performance** across both datasets. We are **interested in how the data differs** for Chloe's **manually collected data and Depop's data**; for df_manual, we are going to isolate Depop sales only. These steps remove that extra "noise" so the two data sources are directly comparable.

(a) Check what platforms exist in the 'PLATFORM' column

```
In [7]: # Check unique inputs in PLATFORM
print(df_manual['PLATFORM'].unique())

['DEPOP' 'POSHMARK' 'MERCARI' 'VESTIAIRE' 'DONATE' 'OTHER' 'VINTED'
'CONSIGN' 'THREDUP' 'EBAY' 'ETSY' nan]
```

(b) Filter to Depop only + drop PLATFORM column

```
In [8]: # keep only "DEPOP" in PLATFORM
rows_before = df_manual.shape[0]
df_manual = df_manual[df_manual['PLATFORM'] == "DEPOP"].reset_index(drop=True)
rows_after = df_manual.shape[0]
print("We removed ", rows_before-rows_after, " rows of data!")

# drop column
df_manual = df_manual.drop('PLATFORM', axis = 1)
print("PLATFORM still in df_manual?", "PLATFORM" in df_manual)
```

We removed 1236 rows of data!
PLATFORM still in df_manual? False

(b) Get a sense of what columns exist in each dataset

```
In [9]: # inspect data
print("Manual data columns:\n", df_manual.columns.values)
print("Depop data columns:\n", df_depop.columns.values)
```

Manual data columns:

```
[ 'SOLD DATE' 'SZN' 'YR' 'BRAND' 'DESCRIPTION' 'TYPE' 'SUBTYPE'
'PURCHASE $' 'DEPOP CATEGORY' 'FLOOR $' 'DEPOP BRAND' 'DEPOP CONDITION'
'SIZE' 'COLOR 1' 'COLOR 2' '(SOLD@ $ + ' 'SHIP $ =' 'PAID $)'
'SHIP+BOOST FEE' 'PLATFORM FEE' 'PAYMENT FEE'
'REVENUE (after ship + fees)' 'PROFIT' 'BOOST' 'SOURCE TYPE' 'SOURCE'
'BUNDLED' 'ORDER #' 'BUY MTH' 'MTHS TO SELL' 'SKU' 'Notes']
```

Depop data columns:

```
[ 'Date of sale' 'Time of sale' 'Date of listing' 'Bundle' 'Buyer' 'Brand'
'Description' 'Size' 'Item price' 'Buyer shipping cost' 'Total'
'USPS Cost' 'Depop fee' 'Depop Payments fee' 'Buyer Marketplace Fee'
'Boosting fee' 'Payment type' 'Estimated payout date'
'Payout arrival date' 'Category' 'Name' 'Address Line 1' 'Address Line 2'
'City' 'State' 'Post Code' 'Country' 'US Sales tax'
'Refunded to buyer amount' 'Fees refunded to seller' 'Payout id'
'Australia Sales tax' 'Unknown Tax' 'Canada Sales tax']
```

(d) quick check on dataset sizes

```
In [10]: print(f"we have {df_manual.shape[0]} rows of manual collected data and {df_depop.shape[0]} rows of depop data")
```

we have 6735 rows of manual collected data and 6453 rows of depop data

2.2.1 Remove columns & unwanted features

Below, we removed columns that:

- are **nominal (ie. SKU or notes)** and don't add meaningful information for our analysis.
- have too many **missing (NaN) values** → we set a **threshold of 50%**.
 - We chose 50% because our dataset is large (thousands of rows), so even if we drop columns with over half missing data, we'll still have 2,000–4,000 complete rows to work with.

The remaining data is still large enough if we just use for the rows with non NaN values, and we can still make interesting analysis and observations in the EDA section.

(a) Observe current columns before deciding what to drop

```
In [11]: # observe the column names again
print("Manual data columns:\n", df_manual.columns.values)
print("Depop data columns:\n", df_depop.columns.values)
```

Manual data columns:

```
[ 'SOLD DATE' 'SZN' 'YR' 'BRAND' 'DESCRIPTION' 'TYPE' 'SUBTYPE'
'PURCHASE $' 'DEPOP CATEGORY' 'FLOOR $' 'DEPOP BRAND' 'DEPOP CONDITION'
'SIZE' 'COLOR 1' 'COLOR 2' '(SOLD@ $ + ' 'SHIP $ =' 'PAID $)'
'SHIP+BOOST FEE' 'PLATFORM FEE' 'PAYMENT FEE'
'REVENUE (after ship + fees)' 'PROFIT' 'BOOST' 'SOURCE TYPE' 'SOURCE'
'BUNDLED' 'ORDER #' 'BUY MTH' 'MTHS TO SELL' 'SKU' 'Notes']
```

Depop data columns:

```
[ 'Date of sale' 'Time of sale' 'Date of listing' 'Bundle' 'Buyer' 'Brand'
'Description' 'Size' 'Item price' 'Buyer shipping cost' 'Total'
'USPS Cost' 'Depop fee' 'Depop Payments fee' 'Buyer Marketplace Fee'
'Boosting fee' 'Payment type' 'Estimated payout date'
'Payout arrival date' 'Category' 'Name' 'Address Line 1' 'Address Line 2'
'City' 'State' 'Post Code' 'Country' 'US Sales tax'
'Refunded to buyer amount' 'Fees refunded to seller' 'Payout id'
'Australia Sales tax' 'Unknown Tax' 'Canada Sales tax']
```

(b) Inspect missing values (NaN) across columns

Here we loop through all columns in both datasets and **calculate the % of missing values (NaN)**. This helps us:

- Identify features that are too incomplete to use reliably
- Set up a rule (50% missing cutoff) for what we'll drop
- Note which columns might still be useful despite partial missing data (like Purchase \$)

We print the results to visually confirm which fields have poor completeness in each dataset.

```
In [12]: print("Columns with >10% NaN in df_manual:")
print('-----')
# Calculate into percentage
for col in df_manual.columns:
    nan_pct = (df_manual[col].isna().sum() / df_manual.shape[0] * 100)
    if nan_pct > 10:
        print(f"{col:30s}: {nan_pct:6.2f}%")

print('-----')

print("Columns with >10% NaN in df_depop:")
print('-----')
# Calculate into percentage
for col in df_depop.columns:
    nan_pct = (df_depop[col].isna().sum() / df_depop.shape[0] * 100)
    if nan_pct > 10:
        print(f"{col:30s}: {nan_pct:6.2f}%")
```

Columns with >10% NaN in df_manual:

```
-----
COLOR 2                : 80.04%
BOOST                  : 58.10%
BUNDLED                : 63.47%
ORDER #               : 92.22%
BUY MTH               : 43.99%
MTHS TO SELL          : 43.99%
SKU                   : 100.00%
Notes                 : 99.27%
-----
```

Columns with >10% NaN in df_depop:

```
-----
Estimated payout date   : 38.65%
Payout arrival date     : 39.11%
Address Line 2          : 90.19%
US Sales tax            : 42.91%
Refunded to buyer amount : 94.82%
Fees refunded to seller : 94.82%
Payout id               : 56.05%
Australia Sales tax     : 97.66%
Unknown Tax             : 96.87%
Canada Sales tax        : 99.07%
-----
```

(c) Drop high-NaN / low-signal columns and set PURCHASE \$ NaNs → 0

We identify and drop some columns with high NaN percentages or irrelevant columns that don't help our financial/marketing goals and standardize purchase cost in manual data.

df_manual

- **PURCHASE \$** – keep / fill NaN → 0

Missing values come from old closet items or freebies. We treat them as \$0 to represent zero cost and make markup/margin math possible.

- **FLOOR \$** – drop

Personal reference floor price used by Chloe; not needed for any regression or summary. Multiplies purchase price by consistent markup multiple to protect for minimum 30% profit margin. Just used for reference as actual listing prices vary.

- **BUNDLED** – drop

Manual bundle flag doesn't match Depop's single-line bundle structure. We'll handle bundles later in EDA.

- **ORDER #** – drop

Nominal source order ID for ThredUp purchases, not analytically meaningful.

- **BUY MTH , MTHS TO SELL** – drop

Rough, manually estimated timing variables. We'll recompute precise time-to-sell from Depop's `Date of listing` and `Date of sale`.

- **COLOR 2 , SKU , Notes** – drop

Sparse or mostly text-based; not relevant to profit or marketing analyses.

Color 2 --> secondary color, mostly NaN since most items are only one color

SKU --> empty column not set up yet and not relevant

Notes --> nominal, mostly NaN, personal reference column

- **BRAND** – drop

Redundant with cleaned accurate `DEPOP BRAND` column. 'Brand' includes specific accurate brands but we will only look at Depop Brand so that random vintage and niche brands will be counted as 'Other'. This enables us to treat vintage/random brands as one category.

- **DEPOP CONDITION** – drop

Not relevant, only used for bulklisting purposes. Most items are "Like new" condition anyways.

- **SIZE** – drop

Nearly constant for Chloe's shop (mostly one size range); adds no variance to models.

- **SHIP \$, PAID \$) , SHIP+BOOST FEE , PLATFORM FEE , PAYMENT FEE** – drop

These raw transaction components are already implied in cleaned numeric fields like ROI and profit margin.

- boost

df_depop

- **Address Line 1 and Address Line 2** – drop

Contain specific buyer street addresses. We already have `city` , `state` , and `country` , which are sufficient for geographic analysis.

- **Name** – drop

Personally identifiable buyer name, nominal only, not relevant for marketing or timing metrics.

- **Payout id** – drop

Nominal identifier for payout batches; not analytically useful.

- **Refunded to buyer amount** This is not part of our research focus, we would drop this column, but we will do it in the later section because we need this to drop rows where 'Refunded to buyer amount' was > 20% of 'Paid'.

- **Fees refunded to seller , and tax columns (Australia Sales tax , Canada Sales tax , Unknown Tax)** – drop

Extremely sparse; not part of our research focus.

- **Buyer shipping cost , USPS Cost** – drop

Almost constant (standard Depop rate); provides no variation for modeling.

Internal shipping label charge; not relevant to buyer behavior or marketing questions.

- **Depop fee , Depop Payments fee , Buyer Marketplace Fee** – drop

Platform transaction fees already accounted for in `payout_amount` . Keeping them would duplicate information.

- **Payment type** – drop

Describes how the buyer paid (PayPal, Depop Payments, etc.). Doesn't affect listing visibility or pricing, so we remove it.

- **Estimated payout date** – drop

Depop's automated estimate of when funds would arrive. Irrelevant.

- **Payout arrival date** – drop

Date funds hit Chloe's account — not meaningful for buyer or listing analysis, since it reflects processing delays rather than marketing or sell through rates.

- **US Sales tax** – drop

Platform-calculated tax amount for certain U.S. states. Mostly 0 or system-generated, not relevant to sales performance or buyer behavior.

```
In [13]: #fill NaN values in purchase price
df_manual['PURCHASE $'] = df_manual['PURCHASE $'].fillna(0)

# drop columns from manual dataset
manual_drop = ['BRAND', 'COLOR 2', 'SKU', 'BUNDLED', 'Notes', 'BOOST',
               'FLOOR $', 'ORDER #', 'BUY MTH', 'MTHS TO SELL',
               'DEPOP CATEGORY', 'DEPOP CONDITION',
               'SIZE', 'SHIP $ =', 'PAID $)', 'SHIP+BOOST FEE', 'PLATFORM FEE',
               'PAYMENT FEE']
df_manual = df_manual.drop(columns=manual_drop, axis=1)

# drop columns from depop dataset
depop_drop = ['Address Line 2', 'Fees refunded to seller',
              'Australia Sales tax', 'Unknown Tax', 'Size',
              'Canada Sales tax', 'Name', 'Payout id',
              'Address Line 1', 'Buyer shipping cost', 'USPS Cost',
              'Depop fee', 'Depop Payments fee', 'Buyer Marketplace Fee',
```

```

        'Payment type', 'Estimated payout date',
        'Payout arrival date', 'US Sales tax']
df_depop = df_depop.drop(columns=depop_drop, axis=1)

print("Filled NaN in 'PURCHASE $' with 0 (if present).")
print("Dropped irrelevant/ nominal columns from both datasets.")
print("Check shapes after drop - manual:", df_manual.shape, "| depop:", df_depop.shape)

```

Filled NaN in 'PURCHASE \$' with 0 (if present).
Dropped irrelevant/ nominal columns from both datasets.
Check shapes after drop - manual: (6735, 14) | depop: (6453, 16)

(d) check columns again

```

In [14]: # Verify changes
print("Manual data columns:\n",df_manual.columns.values)
print("Depop data columns:\n", df_depop.columns.values)

Manual data columns:
['SOLD DATE' 'SZN' 'YR' 'DESCRIPTION' 'TYPE' 'SUBTYPE' 'PURCHASE $'
'DEPOP BRAND' 'COLOR 1' '(SOLD@ $ +' 'REVENUE (after ship + fees)'
'PROFIT' 'SOURCE TYPE' 'SOURCE']
Depop data columns:
['Date of sale' 'Time of sale' 'Date of listing' 'Bundle' 'Buyer' 'Brand'
'Description' 'Item price' 'Total' 'Boosting fee' 'Category' 'City'
'State' 'Post Code' 'Country' 'Refunded to buyer amount']

```

- The 'Buyer' column is nominal data, but it allows us to know if the buyer has bundled items since the Bundle section only indicates bundled or not, the only way to tell which items are bundled together are through Buyer names that are repeated in the same row, Thus we are keeping the nominal buyer column.

2.2.2 Normalize naming of columns

We normalize the names of the columns to explicit and lowercase to make it easier for us to access them later

```

In [15]: # Rename columns to clear, explicit names
rename_map_manual = {
    "SOLD DATE": "sold_date",
    "SZN": "season",
    "YR": "year",
    "DESCRIPTION": "description",
    "TYPE": "item_type",
    "SUBTYPE": "item_subtype",
    "DEPOP BRAND": "brand_listed",
    "PURCHASE $": "purchase_price",
    "COLOR 1": "primary_color",
    "(SOLD@ $ +)": "sold_price",
    "REVENUE (after ship + fees)": "revenue",
    "PROFIT": "profit",
    "SOURCE TYPE": "source_type",
    "SOURCE": "actual_source"
}

rename_map_depop = {
    'Date of sale': "sold_date",
    'Time of sale': "sold_time",
    'Date of listing': "listing_date",
    'Bundle': "bundle",
    'Buyer': "buyer",
    'Brand': "brand",
    'Description': "description",
    'Item price': 'item_price',
    'Total': 'total_price',
    'Boosting fee': 'boosting_fee',
    'Category': 'category',
    'City': 'city', 'State': 'state', 'Post Code': 'postal_code',
    'Country': 'country'
}

# rename_map_depop
df_manual = df_manual.rename(columns=rename_map_manual)
df_depop = df_depop.rename(columns=rename_map_depop)
print(f"Current manual columns ({len(df_manual.columns)}):\n",
      df_manual.columns.values)
print(f"Current depop columns ({len(df_depop.columns)}):\n",
      df_depop.columns.values)

```

Current manual columns (14):
['sold_date' 'season' 'year' 'description' 'item_type' 'item_subtype'
'purchase_price' 'brand_listed' 'primary_color' 'sold_price' 'revenue'
'profit' 'source_type' 'actual_source']
Current depop columns (16):
['sold_date' 'sold_time' 'listing_date' 'bundle' 'buyer' 'brand'
'description' 'item_price' 'total_price' 'boosting_fee' 'category' 'city'
'state' 'postal_code' 'country' 'Refunded to buyer amount']

2.2.3 Preprocessing data

(a) Clean duplicated data

- Chloe is aware that there are duplicate columns for the manual data since she collected it, thus we need to clean that
- The metric that we use is if description, purchase price, sold price, and sold date are all the same then drop the row

```

In [16]: # Remove duplicate rows where description, purchase_price, sold_price, and sold_date are all the same
rows_before = df_manual.shape[0]

df_manual = df_manual.drop_duplicates(
    subset=['description', 'purchase_price', 'sold_price', 'sold_date'],
    keep='first'
).reset_index(drop=True)

print(f"Number of duplicate rows removed: {rows_before- df_manual.shape[0]}")

```

Number of duplicate rows removed: 55

FOR DEPOP DATA - DELETE ROWS WHERE REFUNDED TO BUYER AMOUNT is > 50% of sold price total

this accounts for partially refunded items

```
In [17]: rows_before = df_depop.shape[0]

# Convert both columns to numeric, removing $ and commas
df_depop['Refunded to buyer amount'] = pd.to_numeric(
    df_depop['Refunded to buyer amount'].astype(str).str.replace(r'[\$,]', '', regex=True),
    errors='coerce'
).fillna(0)

df_depop['total_price'] = pd.to_numeric(
    df_depop['total_price'].astype(str).str.replace(r'[\$,]', '', regex=True),
    errors='coerce'
)

# Calculate refund percentage and drop rows
refund_pct = (df_depop['Refunded to buyer amount'] / df_depop['total_price']) * 100
df_depop = df_depop[refund_pct <= 50].reset_index(drop=True)
df_depop = df_depop.drop('Refunded to buyer amount', axis=1)

print(f"Number of rows removed: {rows_before - df_depop.shape[0]}")
```

Number of rows removed: 1084

(a) rows where source type = 'A'

Source Type A indicates item sold for a friend. Does not contribute to Chloe's business financials.

```
In [18]: df_manual = df_manual[df_manual['source_type'] != 'A'].reset_index(drop=True)
```

(b) observe the nan values for df_manual

```
In [19]: df_manual.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6600 entries, 0 to 6599
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   sold_date        6600 non-null   object
1   season           6600 non-null   object
2   year             6600 non-null   float64
3   description       6600 non-null   object
4   item_type        6600 non-null   object
5   item_subtype     6600 non-null   object
6   purchase_price   6600 non-null   float64
7   brand_listed     6595 non-null   object
8   primary_color    5980 non-null   object
9   sold_price       6600 non-null   float64
10  revenue          6600 non-null   object
11  profit           6597 non-null   float64
12  source_type      6600 non-null   object
13  actual_source    6600 non-null   object
dtypes: float64(4), object(10)
memory usage: 722.0+ KB
```

(c) Drop nan values

- We see that for 'brand_listed', 'primary_color', and 'profit', there are some null values, but not a lot, so we are going to drop them.
- for 'boost', we are going to keep them because EXPLAIN EXPLAIN EXPLAIN

```
In [20]: rows_before = df_manual.shape[0]

df_manual = df_manual.dropna(subset=[
    'profit',
    'primary_color',
    'brand_listed']).reset_index(drop=True)

print(f"Number of rows removed: {rows_before - df_manual.shape[0]}")
```

Number of rows removed: 628

(d) doing the same for df_depop, observe and drop nan if necessary

```
In [21]: df_depop.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5369 entries, 0 to 5368
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   sold_date        5369 non-null   object
1   sold_time        5369 non-null   object
2   listing_date     5369 non-null   object
3   bundle          5369 non-null   object
4   buyer            5369 non-null   object
5   brand            5228 non-null   object
6   description       5369 non-null   object
7   item_price       5369 non-null   object
8   total_price      5369 non-null   float64
9   boosting_fee     5369 non-null   object
10  category         5368 non-null   object
11  city             5369 non-null   object
12  state            5369 non-null   object
13  postal_code      5369 non-null   object
14  country          5369 non-null   object
dtypes: float64(1), object(14)
memory usage: 629.3+ KB
```

- We are filling in the NaN values in 'brand' with 'other'
- as well as dropping that 1 row with NaN value in 'category'

```
In [22]: df_depop = df_depop.dropna(subset=['category']).reset_index(drop=True)
df_depop['brand'] = df_depop['brand'].fillna('Other')
```

(e) Converting to time date format

We are converting all the dates into datetime format so we can use them later easily

```
In [23]: # Convert date columns to datetime format
df_manual['sold_date'] = pd.to_datetime(df_manual['sold_date'], format='%m/%d/%y', errors='coerce')

df_depop['sold_date'] = pd.to_datetime(df_depop['sold_date'], errors='coerce')
df_depop['sold_time'] = pd.to_datetime(df_depop['sold_time'], format='%I:%M %p', errors='coerce').dt.time
df_depop['listing_date'] = pd.to_datetime(df_depop['listing_date'], errors='coerce')
```

(f) convert to numeric values

we are converting item_price, total_price, boosting_fee, postal code into numerical values so we can manipulate them later on.

```
In [24]: col_names = ['item_price', 'total_price', 'boosting_fee']
for col in col_names:
    df_depop[col] = pd.to_numeric(
        df_depop[col].astype(str).str.replace(r'[\$,]', '', regex=True),
        errors='coerce'
    )
# for postal code, use the first 5 digits if it contains a dash, else use as is
df_depop['postal_code'] = df_depop['postal_code'].astype(str).str.extract(r'(\d{5})').astype(float)
df_depop[['item_price', 'total_price', 'boosting_fee', 'postal_code']]
```

```
Out [24]:
```

	item_price	total_price	boosting_fee	postal_code
0	24.0	32.15	NaN	33172.0
1	50.0	61.76	NaN	76109.0
2	7.5	12.55	NaN	10013.0
3	50.0	61.04	NaN	26851.0
4	10.0	16.52	NaN	37086.0
...	...	...	...	...
5363	51.0	55.49	NaN	8234.0
5364	35.0	43.47	NaN	95403.0
5365	30.0	37.74	2.84	6032.0
5366	70.0	74.49	5.60	11249.0
5367	92.0	103.16	NaN	81507.0

5368 rows x 4 columns

(g) lets inspect the datasets again

```
In [25]: df_manual.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5972 entries, 0 to 5971
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   sold_date       5972 non-null   datetime64[ns]
1   season         5972 non-null   object
2   year           5972 non-null   float64
3   description     5972 non-null   object
4   item_type      5972 non-null   object
5   item_subtype   5972 non-null   object
6   purchase_price 5972 non-null   float64
7   brand_listed   5972 non-null   object
8   primary_color  5972 non-null   object
9   sold_price     5972 non-null   float64
10  revenue        5972 non-null   object
11  profit         5972 non-null   float64
12  source_type    5972 non-null   object
13  actual_source  5972 non-null   object
dtypes: datetime64[ns](1), float64(4), object(9)
memory usage: 653.3+ KB
```

```
In [26]: df_depop.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5368 entries, 0 to 5367
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   sold_date       5368 non-null   datetime64[ns]
1   sold_time       5368 non-null   object
2   listing_date    5368 non-null   datetime64[ns]
3   bundle         5368 non-null   object
4   buyer          5368 non-null   object
5   brand          5368 non-null   object
6   description     5368 non-null   object
7   item_price     5368 non-null   float64
8   total_price     5368 non-null   float64
9   boosting_fee    407 non-null    float64
10  category       5368 non-null   object
11  city            5368 non-null   object
12  state          5368 non-null   object
13  postal_code     5270 non-null   float64
14  country        5368 non-null   object
dtypes: datetime64[ns](2), float64(4), object(9)
memory usage: 629.2+ KB
```

2.2.3 adding helper columns

(a) Helper columns for manual

- markup multiple - sold price/ purchase price
- ROI - profit/ purchase price only when purchase price > 0, set to 100 when purcha
- margin_pct = net profit as % of buyer total paid (stable even when purchase price is 0)
- dummy categorical columns for seasons - summer spring winter fall (choose one to be referent)
- add column to indicate got item for free

```
In [27]: # helpers for dates & numbers

# add column for free item
df_manual['bought_for_free'] = (df_manual['purchase_price'] == 0).astype(int)

# make column for markup
df_manual['markup_multiple'] = np.where(
    df_manual['purchase_price'] > 0,
    df_manual['sold_price'] / df_manual['purchase_price'],
    np.nan # set to NaN when purchase = 0
)

# make column for roi
df_manual['roi_pct'] = np.where(
    df_manual['purchase_price'] > 0,
    (df_manual['profit'] / df_manual['purchase_price']) * 100,
    np.nan # set to NaN when purchase = 0
)

# make column for margin
df_manual['margin_pct'] = np.where(
    df_manual['sold_price'] > 0,
    (df_manual['profit'] / df_manual['sold_price']) * 100,
    np.nan
)

season_dummies = pd.get_dummies(df_manual['season'], drop_first=True)
df_manual = pd.concat([df_manual, season_dummies], axis=1)

print(f" - markup_multiple: {df_manual['markup_multiple'].notna().sum()} valid values")
print(f" - roi_pct: {df_manual['roi_pct'].notna().sum()} valid values")
print(f" - margin_pct: {df_manual['margin_pct'].notna().sum()} valid values")
print(f" - Items with purchase_price = 0: {(df_manual['purchase_price'] == 0).sum()}")
print(f" - Items obtained for free: {(df_manual['bought_for_free'] == 1).sum()}")

- markup_multiple: 5151 valid values
- roi_pct: 5151 valid values
- margin_pct: 5884 valid values
- Items with purchase_price = 0: 821
- Items obtained for free: 821
```

(b) Helper column for df_depop

- Making a column for days to sell, for depop data, we want to have helper columns for days to sell, this will help us see how many days it took to sell the item, and visualize connection with other variables later on

```
In [28]: # Add a column for days to sell in df_depop
df_depop['days_to_sell'] = (df_depop['sold_date'] - df_depop['listing_date']).dt.days
df_depop['days_to_sell'].head()
```

```
Out[28]: 0    166
1     14
2     79
3     42
4     42
Name: days_to_sell, dtype: int64
```

(c) Another helper column for df_depop

- Making an indicator column for boosted or not, bool column for boosted, knowing whether or not the item is boosted is in the interest of this research topic because it will allow us to visualize the effect of boosting

```
In [29]: # Add a boolean column 'boosted' to indicate if it is boosted
df_depop['boosted'] = df_depop['boosting_fee'].notna().astype(int)
print(df_depop['boosted'].head())
print(df_depop['boosted'].tail())
```

```
0    0
1    0
2    0
3    0
4    0
Name: boosted, dtype: int64
5363    0
5364    0
5365    1
5366    1
5367    0
Name: boosted, dtype: int64
```

```
In [30]: # confirming final shapes
print("Final shape of df_manual:", df_manual.shape)
print("Final shape of df_depop:", df_depop.shape)
```

```
Final shape of df_manual: (5972, 21)
Final shape of df_depop: (5368, 17)
```

```
In [32]: #export cleaned dataframes to csv
df_depop.to_csv("cleanedData/cleaned_depop_data.csv", index=False)
df_manual.to_csv("cleanedData/cleaned_manual_data.csv", index=False)
```

3. Data description

3.1 What are the observations (rows) and the attributes (columns)?

df_manual Each row represents **one individual Depop sale** manually logged by Chloe between 2020–2025. This dataset focuses on **financial performance**, including item cost, sale price, and profit metrics.

Main types of columns:

- **Item details:** brand, type, subtype, primary_color, condition, size
- **Pricing & costs:** purchase_price, item_sale_price, buyer_shipping_paid, buyer_total_paid

- **Fees:** `shipping_boost_fee`, `platform_fee`, `payment_fee`
- **Performance metrics (derived):** `revenue_after_fees`, `net_profit`, `margin_pct`, `markup_multiple`, `roi_pct`, `profit_per_month`
- **Context:** `season`, `year`, `source_type`, `source`, `sold_date`

df_dep

Each row represents **one completed sale** automatically recorded by Depop's system.
It captures platform-side metrics such as listing dates, buyer details, and payout amounts.

Main types of columns:

- **Item/ listing details:** `brand`, `category`, `description`, `date_of_listing`, `date_of_sale`
- **Transaction info:** `item_price`, `depop_fee`, `payout_amount`, `buyer_shipping_cost`, `boosting_fee`
- **Buyer info:** `buyer_city`, `buyer_state`, `buyer_country`
- **Derived fields:** `days_to_sell`, `boosted`

3.2 Why was this dataset created?

df_manual Chloe originally built this dataset to keep track of her business finances; basically, to know which items were profitable and which sources or categories gave her the best returns. Now we're using it to do a deeper financial analysis of her Depop performance across time and types of items.

df_dep This dataset is auto generated by Depop to give sellers information about their sales and payouts history. We're using it for marketing and buyer analysis which means researching how listing details, timing, brand tagging, and buyers affect sell through rates and sales.

3.3 Who funded the creation of the dataset?

df_manual Created and maintained by Chloe for her reselling business. Manually entered after each sale from purchase/ order confirmations and payout statements.

df_dep Automatically generated by Depop's CSV export feature and downloaded quarterly.
We aggregated multiple exports into one complete dataset.

3.4 What processes might have influenced what data was observed and recorded and what was not?

df_manual This dataset reflects Chloe's evolving record-keeping habits. Early years (2020–2021) had fewer fields, and new columns like `purchase_price`, `shipping_boost_fee`, and `source` were added later as her business grew.

Older entries were sometimes filled in retrospectively using memory or old Depop data.

Captured:

- **Costs & profits:** `purchase_price` (filled with 0 when missing), `item_sale_price`, `buyer_shipping_paid`, `buyer_total_paid`, `revenue_after_fees`, `net_profit`, `roi_pct`
- **Context:** `season`, `source_type`, `sold_date`

Not captured:

- Exact `listing_date` or buyer details (only Depop tracks these).
- Manual timing estimates like `BUY MTH` and `MTHS TO SELL` were dropped — we use Depop's timestamps instead.
- Some nominal fields (`ORDER #`, `SKU`, `NOTES`, `FLOOR $`, `BUNDLED`) were removed during cleaning to eliminate noise.

df_dep

Generated automatically by Depop's platform, so it's consistent and timestamped but limited to what Depop logs.

It does **not** include Chloe's purchase costs or sourcing information.

However, it captures all relevant **buyer-facing and platform-facing data**, making it ideal for marketing and timing analysis.

When bundles were listed, Depop sometimes logged them as a **single line item**, leading to some differences in granularity compared to `df_manual`.

3.5 What preprocessing was done, and how did the data come to be in the form that you are using?**df_manual**

- Filtered to include **only Depop sales** (`PLATFORM == 'DEPOP'`)
- Dropped non-useful or redundant fields: `SKU`, `Notes`, `Order #`, `Floor $`, `Buy Mth`, `Mths to Sell`, etc.
- Replaced missing values in `PURCHASE $` with 0 for freebies or old items.
- Dropped sparse nominal fields (`Color 2`, `Boost`, `Bundled`).
- Standardized and renamed columns (e.g., `color_1` → `primary_color`, `purchase $` → `purchase_price`).
- Parsed all date fields and converted numeric columns to floats.
- Created derived metrics:
`margin_pct`, `markup_multiple`, `roi_pct`, `profit_per_month`, and `months_to_sell`.

df_dep

- Dropped high-NaN or irrelevant columns: `Address Line 2`, `Name`, `Refunded to buyer amount`, `Tax columns`, etc.
- Removed nominal payout identifiers (`Payout id`, `Payment type`).
- Standardized `brand` and `category` text fields for consistency.
- Converted prices and dates to numeric and datetime formats.
- Added derived fields: `days_to_sell`, and `boosted` (based on `boosting_fee > 0`).

3.6 If people are involved, were they aware of the data collection and if so, what purpose did they expect the data to be used for?

df_manual Yes — Chloe collected this data herself for personal business tracking. No external participants were recorded.

df_dep Buyers were automatically included through Depop's sales export. No identifying info is used; buyer names were anonymized for this analysis.

3.7 Where can your raw source data be found, if applicable? Provide a link to the raw data (hosted on Github, in a Cornell Google Drive or Cornell Box).

<https://drive.google.com/drive/folders/1vAAwYit-rMo5xFukKuhM6PSaH0tQr7DW?usp=sharing>

Folder contains Depop exported data in multiple CSV files (separated by quarter) Manually collected data CSV also provided

3.8 Is the dataset self-contained, or does it link to or otherwise rely on external resources (e.g., websites, tweets, other datasets)?

Both datasets are **self-contained**.

They rely only on data recorded by Chloe or exported from Depop — no web scraping, APIs, or external datasets were used.

3.9 Does the dataset contain data that, if viewed directly, might be offensive, triggering, or might otherwise cause anxiety?

df_manual

No — the dataset contains neutral, business-related information (sales, prices, dates).

df_depop

No sensitive content is included. Buyer information is anonymized and stripped of personal identifiers before analysis.

3.10 What do the instances that comprise the dataset represent (e.g., documents, photos, people, countries)?

df_manual

Each row = one **sold listing** with all item characteristics and associated financial metrics.

Key variables include:

- `purchase_price`, `item_sale_price`, `buyer_total_paid`,
- `revenue_after_fees`, `profit`, `margin_pct`, `roi_pct`, `markup_multiple`.

df_depop

Each row = one **completed transaction** on Depop.

Includes listing details (`listing_date` , `date_of_sale`), location info (`buyer_city` , `buyer_country`), and transaction metrics (`item_price` , `depop_fee` , `payout_amount`).

3.11 Is there a label or target associated with each instance?

df_manual Yes - Chloe recorded each item within a bundle as a separate row, assigning each its own `purchase_price`, `item_sale_price`, and `net_profit`. This item-level breakdown is useful for financial analysis but doesn't match Depop's single-line bundle entries exactly.

df_depop When bundles were sold, Depop sometimes recorded them as a single line item with one total `item_price`, and other times as multiple per-item entries. This inconsistency means bundle-level data can't always be perfectly aligned with Chloe's manual breakdown.

Chloe occasionally listed items under a different brand than their true brand to increase visibility (e.g., using a trending label). As a result, `brand` in `df_depop` may reflect marketing strategy rather than product identity.

In []: